COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/21/2026	**DATE SUBMITTED:**	10/21/2026
NAME:	Almirol Joel A Clifford Lee C. Doplayna	**INSTRUCTOR:**	Engr. Jimlord M Quejado

ACTIVITY NO. 13.1 **Parallel Algorithms and Multithrerading**

1. PROCEDURE OUTPUT

OUTPUT:

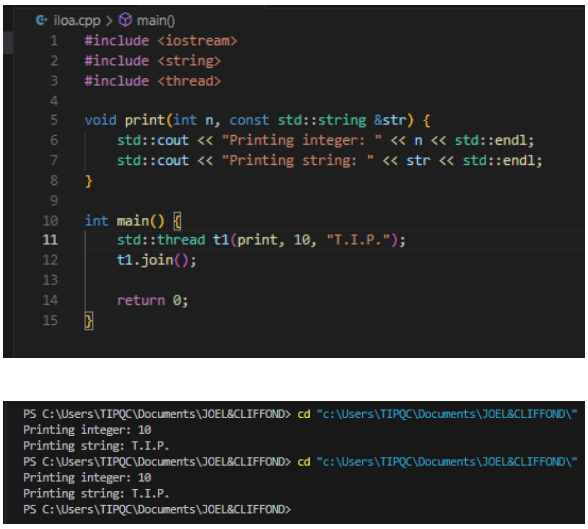
Screenshot	
Analysis	**The main() method starts off by declaring a worker thread called t1 that will take the print() function along with two arguments, which are 10 and "T.I.P." Invoking t1.join() will cause the main thread to wait until t1 has finished printing out an integer and a string in the standard output. This step is important because without the invocation of t1.join(), the main thread will be terminated even before t1 has finished executing and this will lead to std::terminate being called.**

Table 13-1. Simple One-Threaded Example

Screenshot

ilob.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <thread>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

```
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
3 : Engineers0 : T.I.P.
1 : Competent
2 : Computer

PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
1 : Competent
2 : Computer

PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
2 : Computer

PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;

PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
0 : T.I.P.1 : Competent
3 : Engineers
2 : Computer

PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
0 : T.I.P.1 : Competent
3 : Engineers
2 : Computer

PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
3 : Engineers
2 : Computer

PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
2 : Computer

PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;

PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\" ;
0 : T.I.P.3 : Engineers
1 : Competent2 : Computer
```

	
Analysis	**This program exhibits concurrent processing because the process creates multiple worker threads in a loop in which each thread calls the print() function with a unique index and string from the vector s. As the execution of thread is asynchronous and is scheduled dynamically by the operating system, the output in the console shows an example of data race and thread conflict. Since standard output stream operations (std::cout) are not synchronized between different threads, the lines of output will often be printed out of order or mixed up – like 3 : Engineers0 : T.I.P. or 1 : Competent2 : Computer. Storing the created threads in a std::vector<std::thread> and calling th.join() in a loop makes sure that the main thread waits for all concurrent threads to finish their execution to prevent runtime crashing due to calling**

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1.Write a definition of multithreading and its advantages/disadvantages?

Multithreading is a method of execution in C++, whereby a single process spawns multiple threads (std::thread) to run individual tasks concurrently within the same address space. The major benefits of multithreading include maximization of utilization of CPU multi-core capabilities, higher throughput in the application, and maintenance of responsiveness in the application when performing long-running background or I/O tasks. But the disadvantages of multithreading are the increased complexity in coding and overheads incurred during thread creation and management.

2.Rationalize the use of multithreading by providing at least 3 use-cases?

The multithreading capability is necessary in the case of web servers and API services that accept requests from HTTP clients without blocking any new requests. The second scenario where multithreading is widely employed includes GUI and game engines where the main rendering thread is kept responsive, and heavy workloads are done in a separate thread. This is necessary in order to allow the GUI to be responsive without hanging due to the load from heavy work like physics and AI simulation. Finally, the third example includes the case of high-performance computing where data is split up and processed by multiple cores.

3.Differentiate between parallelism and concurrency?

Concurrency is about dealing with multiple tasks at once by overlapping their execution over shared time frames—often on a single processor core through context switching or time-slicing where threads share memory and communicate. In contrast, parallelism is about doing multiple tasks simultaneously across physically separate processing cores or GPUs, typically executing independent copies of code over distinct data sets without requiring interaction between them.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

Code:

Supplymentary > main.Acpp > main()

```cpp
#include <iostream>
#include <thread>

// Global variable of type integer
int global_var = 0;

// Add function that takes an integer parameter and adds it to global_var
void add(int val) {
    global_var += val;
}

int main() {
    // Create 3 threads passing the add function and values
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    // Display state before joining T1
    std::cout << "Value before T1.join(): " << global_var << std::endl;
    T1.join();

    // Display state after T1.join() and before T2.join()
    std::cout << "Value after T1.join(): " << global_var << std::endl;
    T2.join();

    // Display state after T2.join() and before T3.join()
    std::cout << "Value after T2.join(): " << global_var << std::endl;
    T3.join();

    // Final display state after T3.join()
    std::cout << "Final value after T3.join(): " << global_var << std::endl;

    return 0;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND> cd "c:\Users\TIPQC\
Value before T1.join(): 0
Value after T1.join(): 60
Value after T2.join(): 60
Final value after T3.join(): 60
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c
Value before T1.join(): 0
Value after T1.join(): 60
Value after T2.join(): 60
Final value after T3.join(): 60
Value after T2.join(): 60
Final value after T3.join(): 60
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c
Value before T1.join(): 0
Value after T1.join(): 60
Value after T2.join(): 60
Final value after T3.join(): 60
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c
Value before T1.join(): 0
Value after T1.join(): 60
Value after T2.join(): 60
Final value after T3.join(): 60
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c
Value before T1.join(): 0
Value after T1.join(): 60
Value after T2.join(): 60
Final value after T3.join(): 60
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c
Value after T1.join(): 60
Value after T2.join(): 60
Final value after T3.join(): 60
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c
Value after T2.join(): 60
Final value after T3.join(): 60
```

Analysis:

This example shows how calling the join() function is a barrier for synchronization of the main executing thread and other worker threads. As the threads T1, T2, and T3 are initiated together, they operate parallelly in the background and start altering the global variable asynchronously right away. Showing the value of the global variable without calling the T1.join() function could show an intermediate value of the global variable based on the operating system's thread scheduling process because threads might have already completed execution even before printing takes place. Calling the T1.join(), T2.join(), and T3.join() functions serially makes the main thread pause and wait until the particular thread completes its execution.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Code:

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <algorithm>

int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

void parallel_quick_sort(std::vector<int>& arr, int low, int high, int depth = 0) {
    if (low < high) {
        int pi = partition(arr, low, high);

        // Limit recursive thread creation to depth < 2 to manage overhead
        if (depth < 2) {
            std::thread left_thread(parallel_quick_sort, std::ref(arr), low, pi - 1, depth + 1);
            std::thread right_thread(parallel_quick_sort, std::ref(arr), pi + 1, high, depth + 1);
            left_thread.join();
            right_thread.join();
        } else {
            parallel_quick_sort(arr, low, pi - 1, depth + 1);
            parallel_quick_sort(arr, pi + 1, high, depth + 1);
        }
    }
}

int main() {
    std::vector<int> arr = {64, 34, 25, 12, 22, 11, 90, 88, 45, 50};

    std::cout << "Unsorted Array: ";
    for (int x : arr) std::cout << x << " ";
    std::cout << std::endl;

    parallel_quick_sort(arr, 0, arr.size() - 1);

    std::cout << "Sorted Array:   ";
    for (int x : arr) std::cout << x << " ";
    std::cout << std::endl;

    return 0;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary\"
Unsorted Array: 64 34 25 12 22 11 90 88 45 50
Sorted Array:   11 12 22 25 34 45 50 64 88 90
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary\"
Unsorted Array: 64 34 25 12 22 11 90 88 45 50
Sorted Array:   11 12 22 25 34 45 50 64 88 90
PS C:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary> cd "c:\Users\TIPQC\Documents\JOEL&CLIFFOND\Supplymentary\"
Unsorted Array: 64 34 25 12 22 11 90 88 45 50
Sorted Array:   11 12 22 25 34 45 50 64 88 90
```

Analysis:

The use of multithreading in Quick Sort increases the speed of sorting through parallel execution of partition operations for the left and right sub-vectors via separate threads. Because each invocation of a recursive function works on different and mutually exclusive index spans of the vector, the threads operate in isolation from each other without any data race issues or need for mutexes. Thread creation should be limited to the level of recursion depth when it is less than two to avoid overhead.

Conclusion:

Almirol:

By means of this practical session, I acquired an insight into concurrent programming and multithreading in C++ using the thread library. By following the experimental procedures, it was seen that the execution of threads and the synchronization of these threads using std::thread::join() helped in avoiding any early termination of the process and ensured that the workers completed their task execution properly before the main thread exits. The study of the multi-threaded sample code revealed how unsynchronized access to the common output stream (std::cout) resulted in a race condition and thread contention. Moreover, the additional exercise provided me with the practical application of multithreading to sort quickly through parallel sub-array partitioning and overhead reduction through depth limitation.

Doplayna:

After finishing this activity I learned how to create and manage multiple threads in C++ using std::thread and std::thread::join() so programs run faster without crashing. I also learned that when multiple threads try to edit standard output or global variables at the same time, it causes race conditions and messy outputs. The things I need to improve for this activity is I need to make sure vectors are passed by reference so the parallel QuickSort algorithm actually sorts the original data, use std::mutex to lock shared resources, and auto-detect system CPU cores to manage thread count so I can create a much better output

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.